OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

PES UNIVERSITY

OBJECT ORIENTED SOFTWARE ENGINEERING

Behavioral Models
(b) Sequence Models

Archana A
Department of Computer Applications

Sequence Models

- The Sequence model **elaborates the theme of use cases.**
- There are two kind of sequence models:-
  - **Scenarios.**
  - **Sequence diagrams** more structured format.

1. Scenario:

- A scenario is a sequence of events that occurs during one particular execution of a system, such as for a use case.
- A scenario can be displayed as a list of text statements

OBJECT ORIENTED SOFTWARE ENGINEERING

Scenario for the session with Online Stock Broker

John Doe logs in.
System establishes secure communications.
System displays portfolio information.
John Doe enters a buy order for 100 shares of GE at the market price.
System verifies sufficient funds for purchase.
System displays confirmation screen with estimated cost.
John Doe confirms purchase.
System places order on securities exchange.
System displays transaction tracking number.
John Doe logs out.
System establishes insecure communication.
System displays good-bye screen.
Securities exchange reports results of trade.

Scenario

- At early stages of development, Scenarios are expressed at a high level, later stages we can show exact messages.
- First step of writing a scenario is to identify the objects exchanging messages.
- Determine the sender and receiver of each message and sequence of messages.
- Add activities for internal computations.

2. Sequence Diagram:

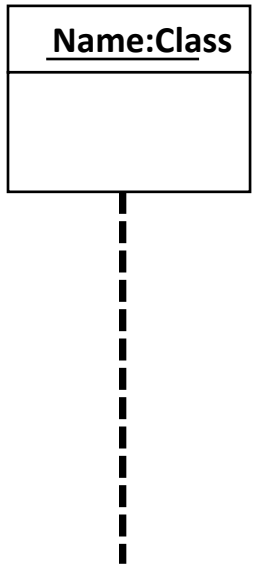
- A sequence diagram shows the **participants** in an interaction and the **sequence of messages** among them.

Components of Sequence diagram

1. Object:

- Objects are represented by rectangles and **name of the objects are underlined**

- Object life line are denoted as **dashed lines**. They are used to model the existence of objects over time



Sequence diagram…

- Each actor as well as the system is represented by a vertical line called lifeline and each message by a horizontal arrow from sender to the receiver.

- Each use case requires one or more sequence diagram to describe its behavior.

- For large scale interactions containing many independent tasks, we can draw separate sequence diagram for each task.

Eg:-

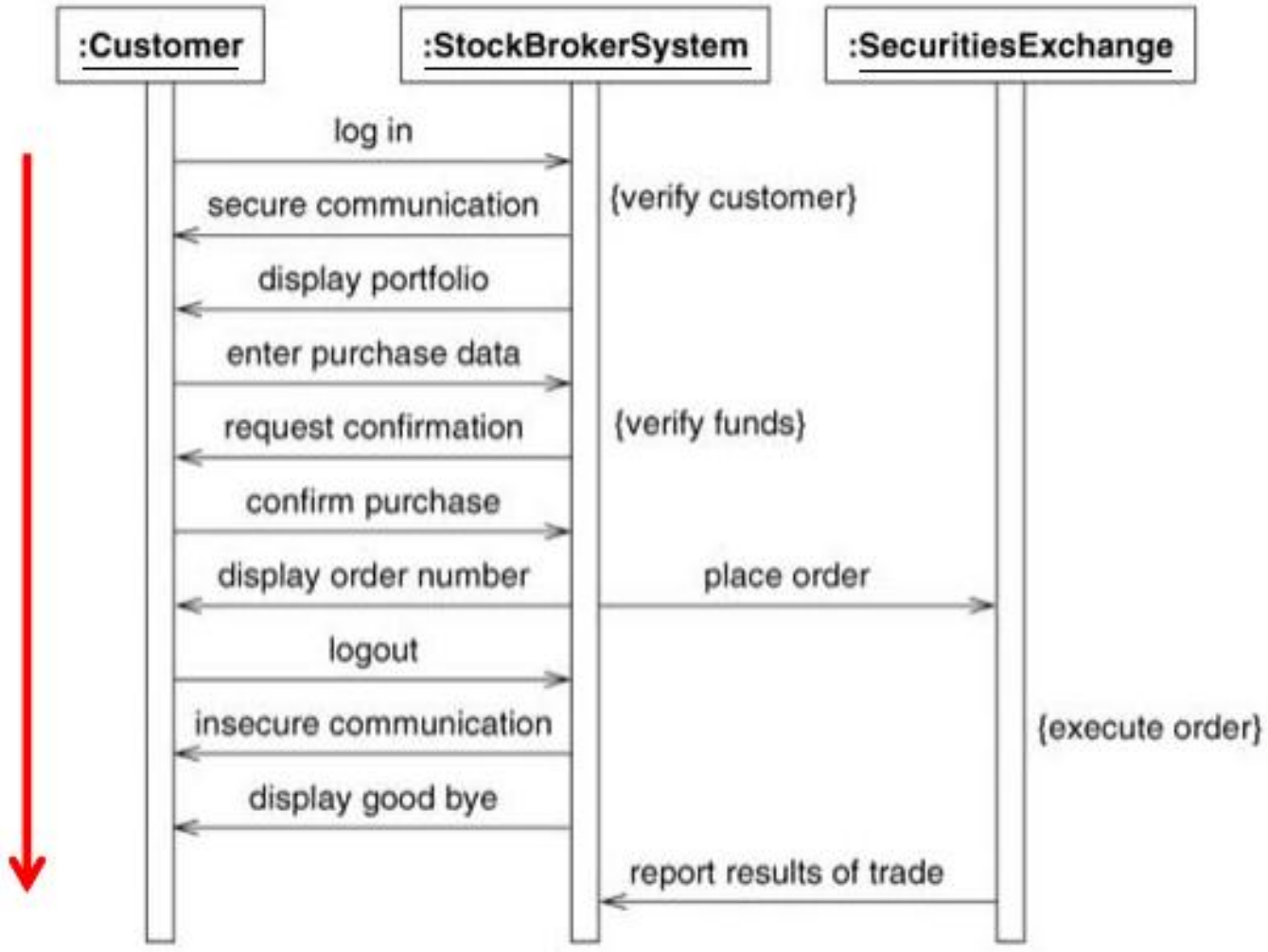
- Sequence diagram for a stock purchase
- Sequence diagram for a stock quote
- Sequence diagram for a stock purchased **that fails**.

Scenario with sequence diagram example

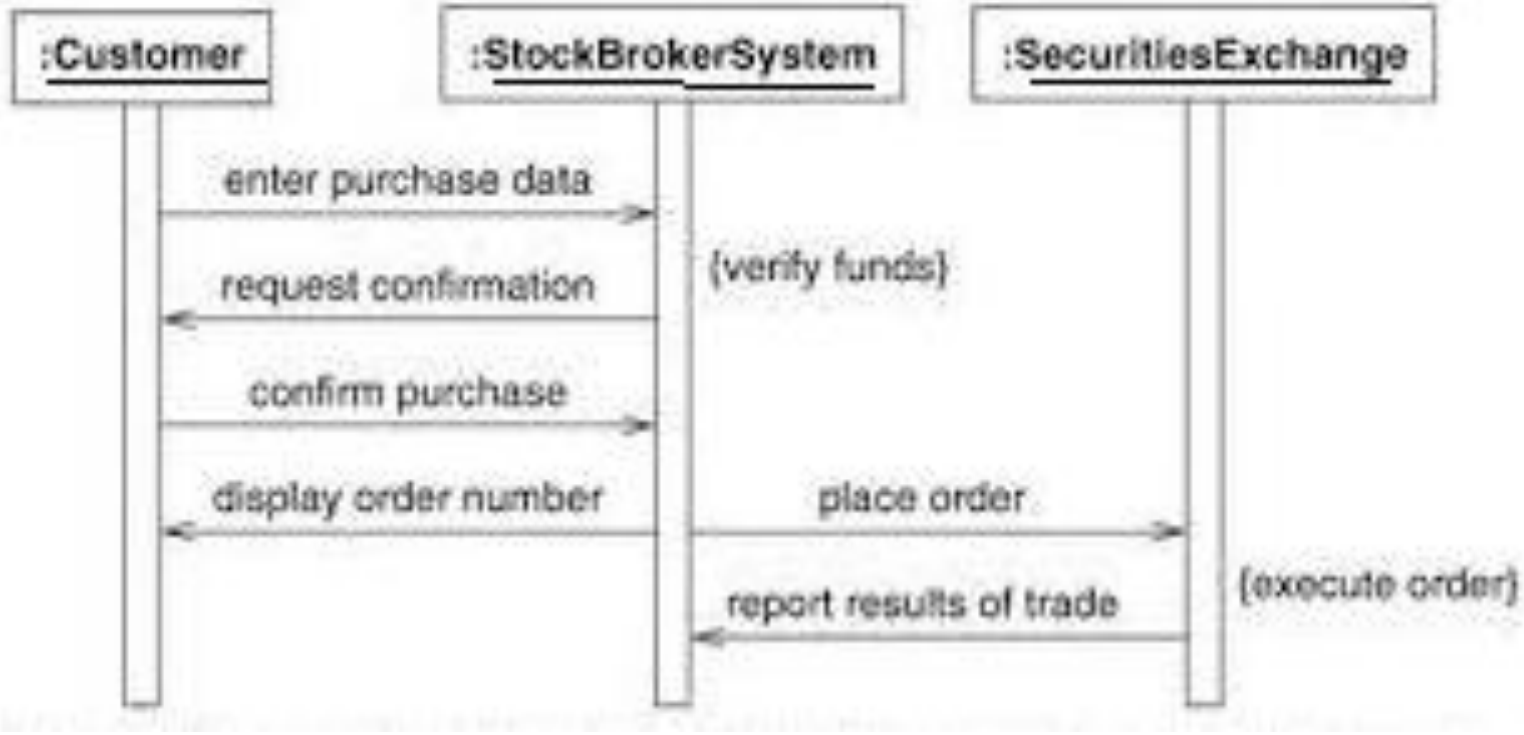
John Doe logs in.
System establishes secure communications.
System displays portfolio information.
John Doe enters a buy order for 100 shares of GE at the market price.
System verifies sufficient funds for purchase.
System displays confirmation screen with estimated cost.
John Doe confirms purchase.
System places order on securities exchange.
System displays transaction tracking number.
John Doe logs out.
System establishes insecure communication.
System displays good-bye screen.
Securities exchange reports results of trade.

OBJECT ORIENTED SOFTWARE ENGINEERING

Scenario with sequence diagram example



Sequence diagram – Stock purchase



Sequence diagram –stock quote

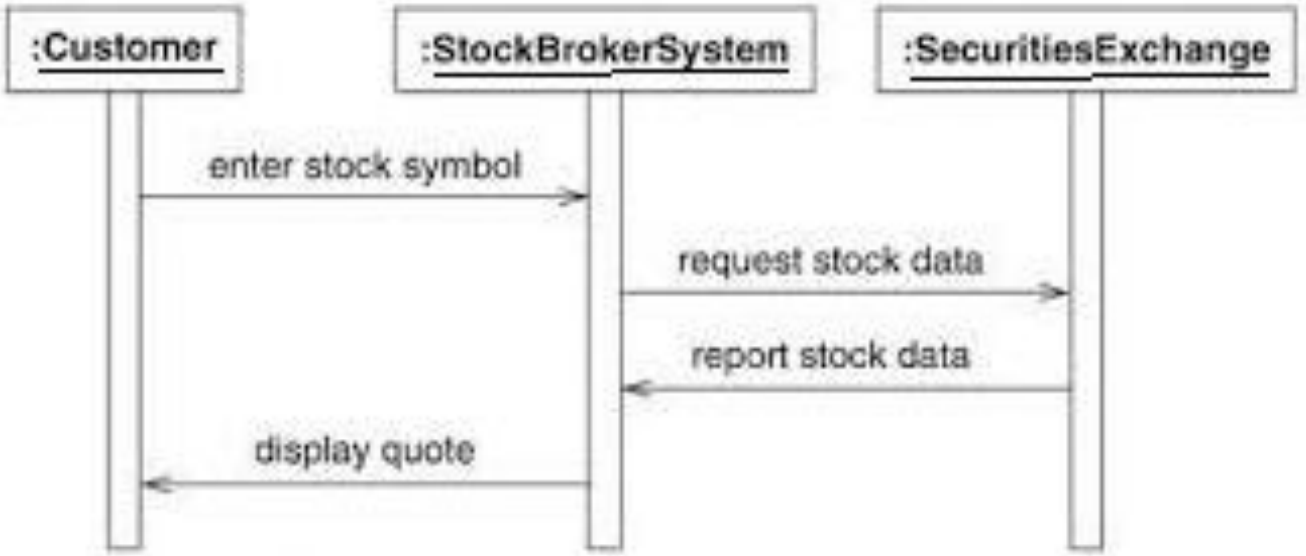


Figure Sequence diagram for a stock quote.

Scenario with sequence for stock purchase that fails

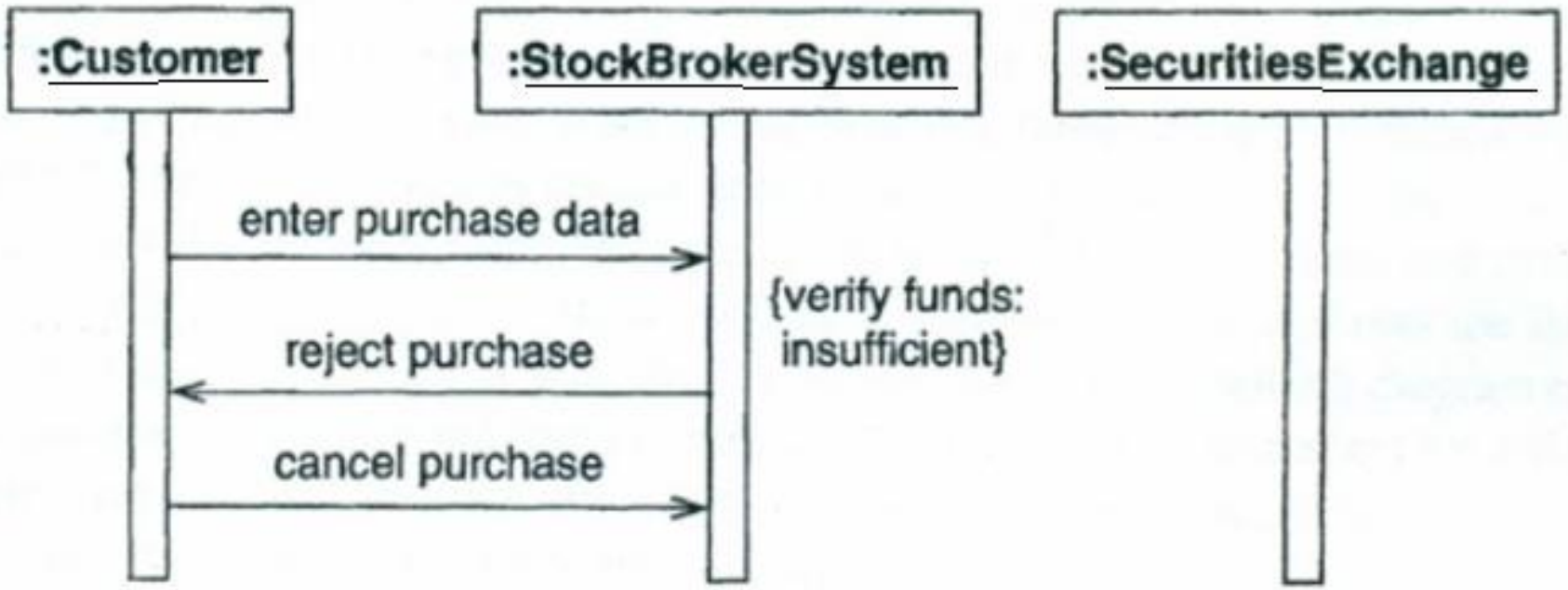


Fig Sequence diagram for a stock purchase that fails

2. Messages:

- In message **square brackets** containing **a guard conditions**. This is a Boolean condition that must be satisfied to enable the message to be sent.

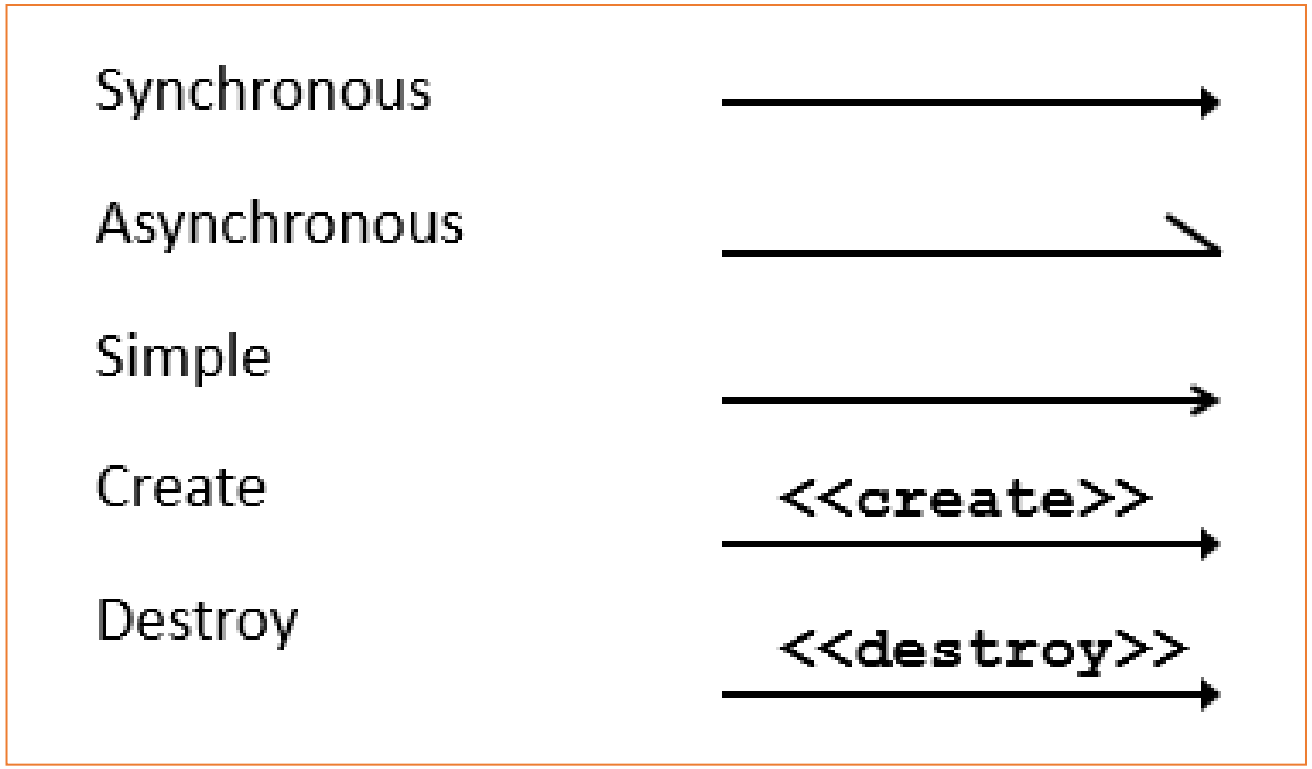
- May have an **asterisk followed by square brackets** containing an **iteration specification**. This specifies the number of times the message is sent.

Components of sequence diagram…

- May have **return list** consisting of a **comma -separated** list of names that designate the values of returned by the operation.
- Must have **a name or identifier** string that represents the message.
- May have **parentheses** containing an **argument list** consisting of a comma separated list of actual parameters passed to a method.

Messages…

Types of Messages

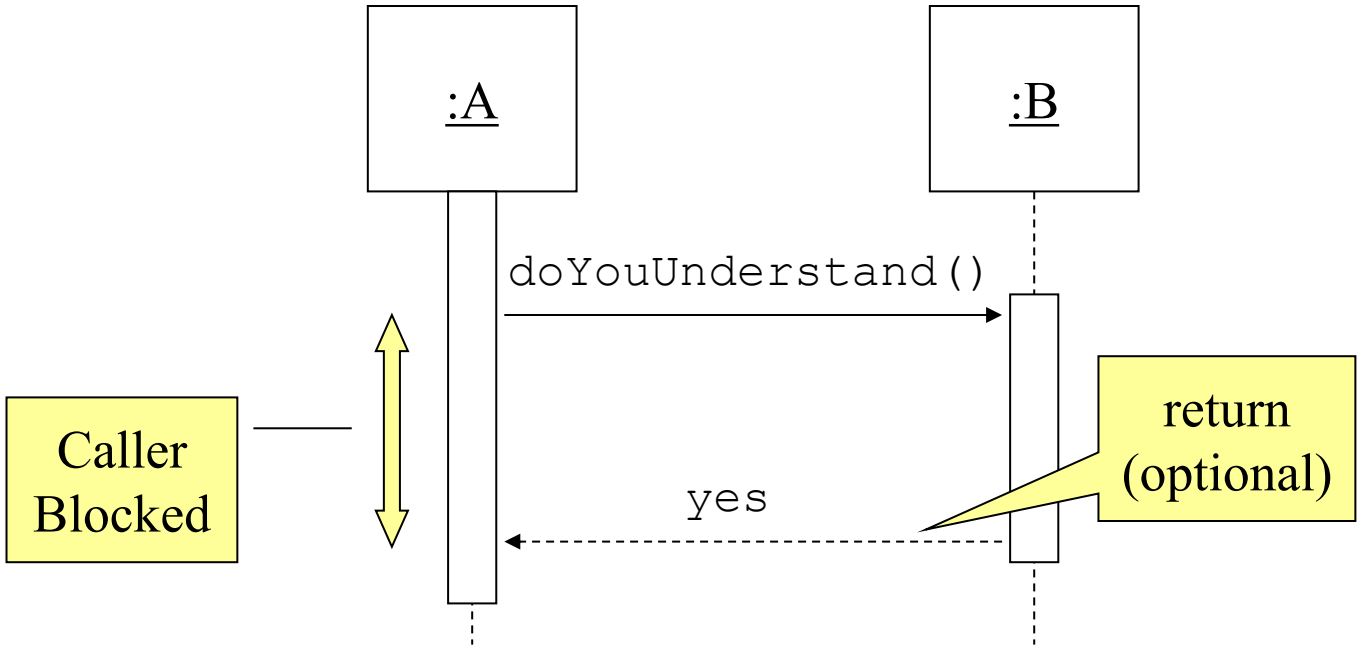


Messages…

Synchronous Messages:

Nested flow of control, typically implemented as an operation call.

The routine that handles the message is completed before the caller resumes execution.



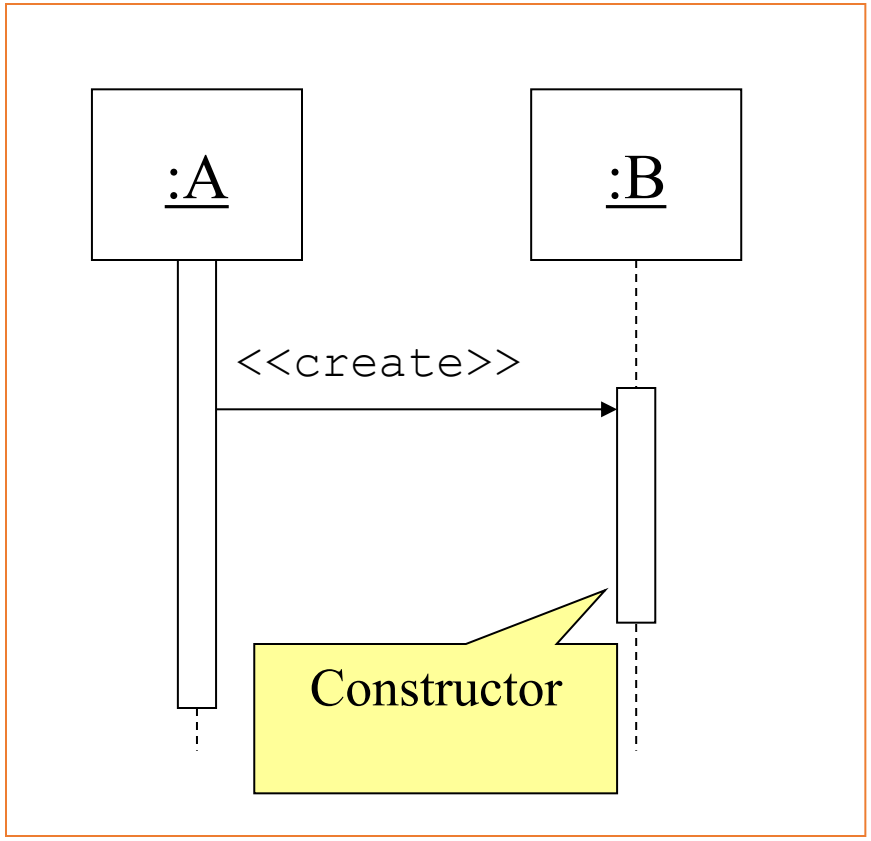
Messages…

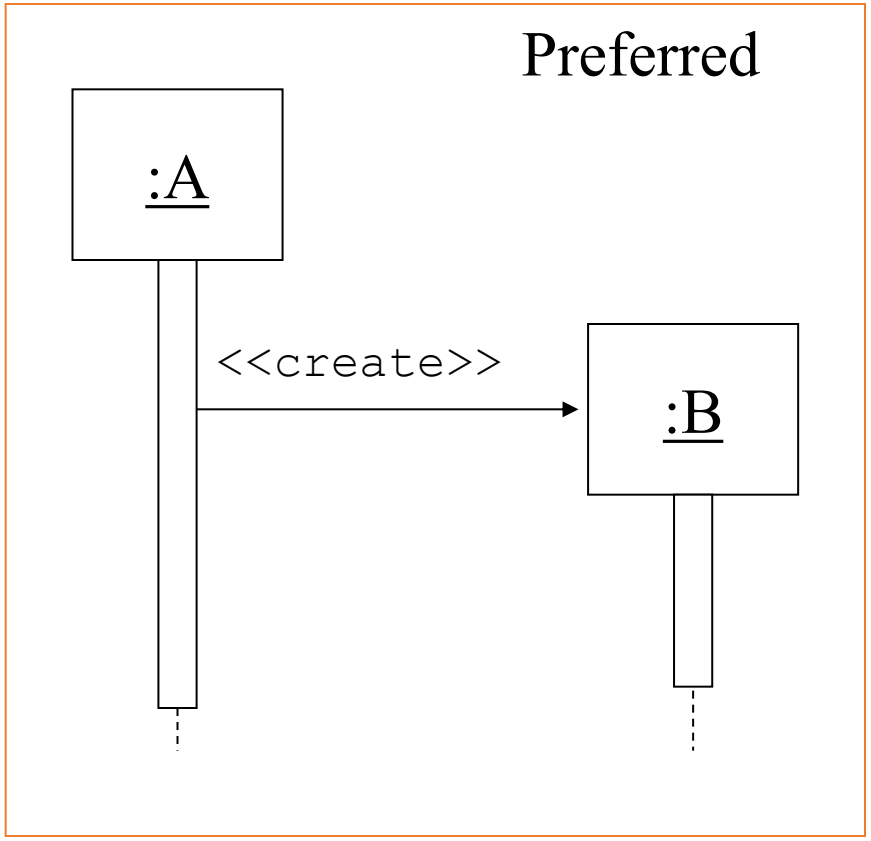
Optionally indicated using a dashed arrow with a label indicating the return value.

- Don’t model a return value when it is obvious what is being returned, e.g. getTotal()
- Model a return value only when you need to refer to it elsewhere, e.g. as a parameter passed in another message.
- Prefer modeling return values as part of a method invocation, e.g. ok = isValid()

Messages…

- **Object Creation:**
- An object may create another object via a **<<create>>** message.

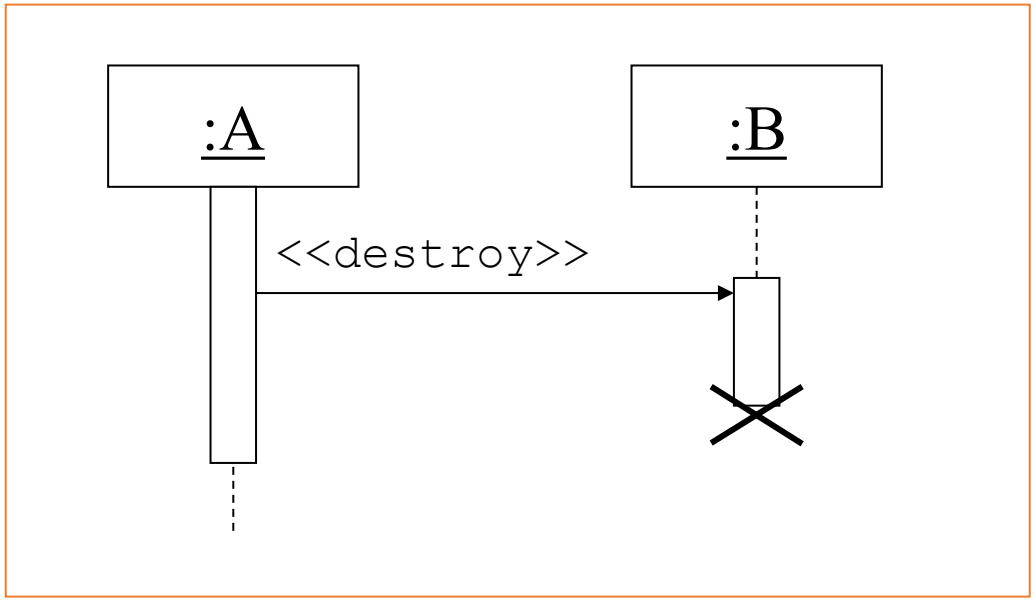




Object Destruction:

An object may destroy another object via a <<destroy>> message.

- An object may destroy itself.
- Avoid modeling object destruction unless memory management is critical.



THANK YOU

Ms. Archana A
Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392